Create Our First Changeset

The functional nature of Elixir is visibly seen in the changeset function we're going to write. Changesets are built up over multiple function calls, which is perfect for the pipeline operator. This is usually done over several stages:

1. Cast the provided attributes to the types of our schema fields.
2. Apply validations on a per-attribute basis.
3. Define unique constraints on the schema.

Add the following code to the bottom of the Schema.Contact module:

phone_app/lib/phone_app/conversations/schema/contact.ex

```
import Ecto.Changeset

def changeset(attrs) do
  fields = [:phone_number]

  %__MODULE__{}
  |> cast(attrs, fields)
  |> validate_required(fields)
  |> unique_constraint([:phone_number])
end
```

Our code starts by importing the Ecto.Changeset module. This allows us to call changeset functions (like cast/3) without typing out the full module name.

Our changeset/1 function consists of a functional pipeline that calls various Ecto.Changeset functions. We start by calling cast/3 with three arguments: an empty schema struct, the input attributes, and the fields that we want to set in the database.

We pass an empty struct because cast/3 is capable of accepting structs with data, which is used to update a record. Passing an empty struct tells the changeset that we want to create a new record.

We call validate_required/2 to set our fields as required. This ensures that a null or empty :phone_number field will cause the changeset to fail. This is one of the many validation functions that Ecto.Changeset implements.

Finally, we call unique_constraint/2 to add information about field uniqueness. The unique_constraint function adds information about the constraint to the changeset, but it doesn't make any database queries. When we insert the changeset in the database, it will intelligently return an error if this constraint is violated.

The fields variable in this code is a convenience that has become my go-to pattern. It's important that required fields are properly defined, so it's convenient to